

Programação Multithread com a biblioteca POSIX Threads

Pedro Arthur Duarte

¹<http://pedroarthur.com.br/>
<http://under-linux.org/blogs/pedroarthurjedi>
pedroarthur jedi [AT] gmail com

1. Introdução

Nos últimos anos temos visto uma enxurrada de novos processadores clamando o uso de múltiplos núcleos. Eles se aproveitam dos avanços das tecnologias de miniaturização de componentes e criam soluções que só eram disponíveis anteriormente com o uso de diversos processadores e da duplicação de diversos outros componentes.

Esses processadores tiram proveito da paralelização da fila de execução. Ou seja, ao invés de estarem processando uma única operação por vez, tais processadores são capaz de dividir as tarefas, cada qual se dedicando a uma operação por vez.

Para tirar proveito dos recursos, as aplicações precisam passar por uma pequena modificação: divisão dos fluxos de execução. Ou, como mais conhecido, as aplicações precisam se tornar *multithreads*.

Ou seja, devemos estudar e analisar as aplicações visando a identificação de pontos passíveis de paralelização. Um exemplo simples disso seria um browser de Internet. As diferentes abas exibem páginas distintas, portanto, o carregamento de cada aba pode ocorrer de forma independente uma da outra. Apesar de existirem diversos tipos de paralelismo, focaremos aqui no modelo de multiprocessadores com memória compartilhada.

Aqui não serão apresentados os fundamentos do processamento *multithread* pois deixaria o texto enfadonho, principalmente para a maioria do pessoal que não possui interesse nem disposição de estar analisando funções matemáticas. Mas, para aqueles que desejam uma abordagem mais voltada a área da pesquisa acadêmica, recomendo o livro *"Introduction to Parallel Computing"* escrito por Grama, Karypis, Kumar e Gupta.

No decorrer do texto veremos como criar aplicações *multithreads* e como contornar os problemas envolvidos. Os exemplos estarão em C mas no futuro mostrarei como criar aplicações *multithread* em Python e Java. Todos os exemplos assumem que você esteja desenvolvendo em um derivado do Linux.

2. A Biblioteca POSIX Threads

A biblioteca pthreads (*POSIX Threads*) é baseada nas especificações POSIX.1 que define um conjunto de tipos, funções e macros relativos a criação e ao controle de diversos fluxos de execução, chamados daqui para frente de *threads*.

No modelo de processamento *multithreads* POSIX, as diversas threads residem no mesmo espaço de endereçamento do processo que as criou, ou seja, compartilham das mesmas variáveis, descritores de arquivo e ainda compartilham o espaço disponível para alocação da pilha do processo. Portanto, mesmo que seu processo tenham 600 *threads*, a alocação de memória estará disponível como se fosse apenas um processo.

Para iniciar o uso de funções da biblioteca *PThreads*, o arquivo fonte deve incluir o cabeçalho *pthread.h*:

```
#include <pthread.h>
```

Além disso, todos os comandos de compilação devem indicar o uso da biblioteca:

```
$ gcc -o saida -lpthread entrada.c
```

Doutra forma, pode esperar um monte de mensagens nada amigáveis do GCC.

2.1. Criando e terminando novas threads

O padrão *PThreads* exige que funções que serão chamadas para a criação de novas *threads* possuam uma assinatura específica. A saber:

```
void* (void *)
```

Ou seja, a função a ser executada precisa obrigatoriamente retornar um ponteiro genérico e receber como parâmetro de entrada um ponteiro genérico. O valor de retorno, como veremos adiante, pode ser recuperado e usado posteriormente. O parâmetro de entrada pode ser usado para passar um dado qualquer para a nova *thread*.

Criada a função com a assinatura supracitada, podemos então usar a função `int pthread_create(pthread_t*, pthread_attr*, void*(*)(void *), void *)`. O primeiro parâmetro receberá o identificador único da *thread*; o segundo parâmetro serve para criar uma *thread* com atributos especiais, os quais veremos mais abaixo; o terceiro parâmetro é a função a ser executada pela *thread*; e o quarto parâmetro é o parâmetro de entrada da função. O valor de retorno de `pthread_create` é zero caso a nova *thread* seja criada com sucesso, ou um valor diferente de zero caso haja algum erro. Recomendo executar `man pthread_create` e dar uma breve lida.

Analisemos o seguinte exemplo:

```
#include <stdio.h>
#include <pthread.h>

void* print(void *data){
    char *str = (char *) data;

    while (1)
        printf("%s ", str);
}

int main (int argc, char** argv) {
    int i;
    pthread_t threads[argc];

    for (i = 1 ; i < argc ; i++)
        pthread_create(&threads[i], NULL, print, (void*) argv[i]);

    return 0;
}
```

Nesse pequeno programa queremos que ela imprima na tela os parâmetros passados ao programa, porém, ao invés de fazer o trabalho em série, será criada uma *thread* para cada parâmetro. Para tal, criamos uma pequena função chamada *print* que atende todos os requisitos do padrão (valor de retorno e parâmetro do tipo *void**).

Como se pode inferir, *pthread_t* é um tipo de dado definido pelo padrão. Sua principal função é armazenar o identificador único de cada *thread*. Esse valor é usado em algumas funções que veremos mais a frente.

Como não temos nenhuma necessidade adicional nessas *threads*, o segundo parâmetro foi definido como nulo e será ignorado pela função *pthread_create*, que criará as *threads* com os atributos padrões.

Assim que a *thread* estiver pronto para executar, o sistema dará lugar a função *print*. Essa fará um *cast* do quarto parâmetro para um *char**. Esse deverá ser impresso tantas vezes quanto possível.

Agora, devemos compilar nosso programa. Assumindo que esse esteja salvo sob o nome *threads.c*:

```
$ gcc -o threads -lpthread -Wall threads.c
```

Agora, executemos nossa pequena aplicação com os parâmetros “under linux org”:

```
pedroarthur@coruscant:~/ccc$ ./a.out under linux org
pedroarthur@coruscant:~/ccc$
```

Putz! Não saiu nada! Mais uma vez:

```
pedroarthur@coruscant:~/ccc$ ./a.out under linux org
org org org org org org org org org org org org org org
org org org org org org org org org org org org org org
org org under under under under under under under u
nder under under under under under under under unde
r under under org under org under org linux linux l
inux linux linux linux linux linux linux linux linu
x under org org org org linux org linux linux linux
org linux linux linux org linux org linux org linux
linux linux org linux linux linux org linux org lin
ux org linux org org org linux org linux org linux
linux org linux linux linux org linux org linux org
pedroarthur@coruscant:~/ccc$
```

Viram que comportamento estranho? Pois é, o uso de *threads* pode causar efeitos colaterais indeterminados a sua aplicação. As *threads* não são organizadas de maneira serial, portanto, não se deve esperar nada de seu comportamento.

Nesse primeiro exemplo, a maioria devia estar esperando que “under” fosse escrito antes de “linux”, e por sua vez esse fosse escrito antes de “org”. Porém, na primeira execução, não tivemos nada na saída. Ou seja, foi dado ao *thread* principal um maior tempo de execução. Então, esse avançou logo para a função de retorno e saiu do programa. Já na segunda execução, as *threads* criadas receberam mais atenção que a *thread* principal

e puderam cada qual escrever algumas vezes na tela. Então, tentando frisar, não se deve esperar nada do comportamento das *threads*! Uma podem avançar mais que outras, ou talvez nem cheguem a executar! Estejam cientes!

Como nota-se pelo evento, as *threads* são finalizadas tão logo a função *main* retorna. Então, acredito que tenha ficado a dúvida de como deixar uma *thread* executando mesmo que se deseje finalizar a função *main*. Um método simples é finalizar a thread em que a função *main* está executando. Em outras palavras, terminar o fluxo de execução sem utilizar a diretiva `return`. Então, bastaria fazer¹:

```
/* inicio do código */

for (i = 1 ; i < argc ; i++)
    pthread_create(&threads[i], NULL, print,
        (void*) argv[i]);
pthread_exit(NULL);

/* Restante do código */
```

A função *pthread_exit* finaliza a *thread* na qual ela é chamada. O parâmetro de *pthread_exit* deve ser substituído pelo valor que se deseja retornar. O valor de retorno pode ser recuperado por outra função da biblioteca *PThread*, que veremos um pouco mais abaixo.

Porém, fazendo a *thread* principal terminar sua execução dessa forma nota-se, mediante consulta na tabela de processos, que será concatenado o pós-fixado "<defunct>":

```
1000      6780 14.0  0.0      0      0 pts/3      Zl+   07:28
0:00 [a.out] <defunct>
```

Ou seja, para o sistema seu processo é um **zumbi**. Apesar de não ser um grande problema, nesse caso, ter um processo nesse estado não é considerado uma boa prática. Portanto, ao invés de finalizar a *thread* principal, é mais interessante aguardar que as outras *threads* finalizem usando a função *pthread_join(pthread_t, void**)*. O primeiro parâmetro é o identificador único da *thread* que se deseja aguardar o fim da execução. O segundo parâmetro é um ponteiro genérico que deverá receber o valor de retorno da *thread* que está sendo aguardada.

Como em nosso exemplo anterior o fluxo de execução das threads não tem fim, vamos atualizar nosso código para o seguinte:

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

void* print(void *data){
    char *str = (char *) data;
    int i;
```

¹Para finalizar a execução pressione *ctrl+c*.

```

    srandom(time(NULL));

    for (i = 0 ; i < random() ; i++)
        printf("%s ", str);

    pthread_exit((void*)i);
}

int main (int argc, char** argv) {
    int i;

    pthread_t threads[argc];
    void *retval[argc];

    for (i = 1 ; i < argc ; i++)
        pthread_create(&threads[i], NULL, print, (void*) argv[i]);

    for (i = 1 ; i < argc ; i++)
        pthread_join (threads[i], &retval[i]);

    printf ("\n");

    for (i = 1 ; i < argc ; i++)
        printf ("%s: %d\n", argv[i], (int)retval[i]);

    return 0;
}

```

Novamente, devemos compilar o código incluindo a biblioteca *pthread*, assumindo o nome do arquivo fonte como *threads.c*:

```
pedroarthur@slackhlbr:~/ccc$ gcc -lpthread -Wall threads.c
```

Rodando nossa aplicação com os parâmetro "the jedi lairs":

```

/* Um monte de repetições das palavras "the", "jedi" e "lair" */
the: 84570
jedi: 54323
lair: 39598
pedroarthur@slackhlbr:~/ccc$

```

A função *print* foi modificada para imprimir um número randômico de vezes a *string* passada como parâmetro (linhas 11 e 12). Colocamos também, no final do código de *print*, a função *pthread_exit* para retornar o valor de *i* após o fim da execução do laço (linha 14). Como o valor de *i* é um inteiro, e a função espera receber um ponteiro genérico, tivemos que fazer um *cast* na variável *i*. Tenham em mente que essa solução não é portátil!!!

Já na função *main*, adicionamos um vetor de ponteiros genéricos para receber os valores de retorna das *threads* (linha 21). Além disso, tivemos que adicionar chamadas

a função *pthread_join* para que as mesmas recebessem os códigos de retorno (linhas 26 e 27). Por fim, adicionamos um pequeno trecho de código para imprimir quantas vezes cada um dos parâmetros da linha de comando foram impressos (linhas 31 e 32).

Uma *thread* que chame a função *pthread_join* ficará bloqueada até que a *thread* que ela está aguardando finalize. Em outras palavras, se temos as *threads* *A* e *B*, e *A* chama *pthread_join(B, NULL)*, ela ficará sem executar nenhuma outra instrução até que *B* chame *pthread_exit(NULL)*.

2.2. Atributos das Threads

Os atributos das *threads* permitem definir determinados comportamentos para essas. Os atributos de uma *thread* podem ser determinados no momento da criação da mesma, com o uso de variável atributo, ou com funções específicas que tem como parâmetro o ID da *thread* a ser manipulada.

Para definir os atributos no momento da criação de uma *thread*, devemos inicializar uma variável de atributos. Essas variáveis e as funções que as manipulam possuem como prefixo *pthread_attr*, sendo o tipo atributo *pthread_attr_t*. O código abaixo mostra a inicialização de uma variável atributo.

```
pthread_attr_t attr;
pthread_t thread;

/* ... */

pthread_attr_init (&attr);
/* Definição de alguns comportamentos especiais */

pthread_create (&thread, &attr, &thread_f, thread_param);
pthread_attr_destroy (&attr);
```

A função *pthread_attr_init* inicializa a variável atributo, definindo seus valores para o padrão. Após criada e ajustados os valores necessários, a variável atributo precisa ser passada como segundo argumento para a função *pthread_create* para ser corretamente utilizada. A função *pthread_attr_destroy* remove todos os parâmetro definidos, evitando assim que um futuro uso descuidado da variável atributo gere problemas.

2.2.1. Separação das Threads

Dentre os comportamentos a serem modificados, o mais frequente deles é a separação da *thread*. Tradução do inglês *detach*, separar uma *thread* significa tornar sua execução e finalização independente das outras *threads*. Quando temos uma *thread* não separada, seu valor de retorno permanece alocando recursos até que alguma outra *thread* recolha-o em meio a chamada *pthread_join*. Numa *thread* separada, os recursos são dealocados automaticamente, como seria feito com um processo que retorna. Contudo, após separada, uma *thread* não mais pode ter seu valor de retorno recuperado por uma chamada a *pthread_join*. Uma atualização do excerto de código mostrado anteriormente demonstra o uso do *pthread_attr_setdetachstate*:

```
pthread_attr_t attr;
pthread_t thread;

/* ... */

pthread_attr_init (&attr);
pthread_attr_setdetachstate (&attr, PTHREAD_CREATE_DETACHED);

pthread_create (&thread, &attr, &thread_f, thread_param);
pthread_attr_destroy (&attr);
```

Ou seja, todas as *threads* que recebam como atributo de criação a variável *attr* irá ser criada separada das demais.

Outra maneira de separar uma *thread* é chamar a função *pthread_detach(pthread_t*)*. O primeiro argumento é a *thread* a qual se deseja separar. Nesse ponto cabe introduzir a função *pthread_self()*. Essa função retorna o identificador da *thread* que a chama. Portanto, caso desejemos em determinado ponto do fluxo de execução separar uma *thread* das demais, fazemos:

```
/* código, código, código... */

pthread_detach(pthread_self());

/* código, código, código... */
```

pthread_detach pode ser chamada por qualquer outra *thread*. Porém, deve-se estar ciente que misturar o fluxo de execução das diversas *threads* pode ter efeitos inesperados.

2.2.2. Escopo de Execução

No *PThreads*, os novos fluxos de execução podem rodar em dois escopos distintos. No primeiro deles, o escopo do sistema, as *threads* irão competir como um processo comum pelos recursos do sistema. Ou seja, caso você tenha um processo com quatro *threads* de escopo de sistema e um outro processo com apenas uma *thread*, o tempo de processamento será dividido como se o sistema operacional estivesse escalonando 5 processo diferentes; Já no escopo do processo, as *threads* irão competir internamente pelo tempo de processador dado ao processo que as criou.

O comportamento descrito acima pode ser definido apenas durante a criação da *thread* usando uma variável atributo e passando-a como parâmetro para a função *pthread_attr_setscope*. Duas macros definem os comportamentos: *PTHREAD_SCOPE_SYSTEM* e *PTHREAD_SCOPE_PROCESS*. O código abaixo apresenta uma atualização da função *main* do nosso primeiro exemplo para que o mesmo utilize-se do escopo do sistema:

```
/* includes e função print */
int main (int argc, char** argv) {
    int i;
```

```

pthread_attr_t attr;
pthread_t threads[argc];

pthread_attr_init(&attr);
pthread_attr_setdetachstate (&attr, PTHREAD_CREATE_DETACHED);
pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

for (i = 1 ; i < argc ; i++)
    pthread_create(&threads[i], &attr, print, (void*)argv[i]);

return 0;
}

```

Apesar de não apresentar uma mudança brusca no comportamento do programa, em situações onde o processamento é mais necessário essa mudança se mostra significativa.

2.2.3. Escalonamento

Escalonar um processo significa agendar tempo de processamento e outros recursos, tais como de entrada/saída, para a execução do processo. Apesar de ser usada a palavra "processos", o conceito também é aplicável as *threads*, bastando apenas fazer uma substituição dos termos.

Os algoritmos de escalonamento são a base da tomada de decisão de qual *thread*/processo irá executar caso hajam mais de uma *thread*/processo pronto para tal. No descrito até agora, a única função que altera o comportamento do escalonador é *pthread_attr_setscope*, a qual define o escopo de execução de uma *thread*.

Além do escopo, existem outros três atributos que definem as tomadas de decisão de escalonamento das *threads*. Um dos mais interessantes são as prioridades das *threads*. Esse atributo permite definir, quando há mais de uma *thread* bloqueada, qual será a próxima a executar. O mecanismo é simples: a *thread* de maior prioridade possui maior precedência.

As políticas de escalonamento permitem a definição de diferentes algoritmos para o controle da ordem de execução das *threads*, aproveitando-se também das prioridades definidas pela função *pthread_attr_setschedparam*. A primeira política, a *First-in First-out*, leva o escalonador ao comportamento quem chegar primeiro leva. Ou seja, dada duas *threads* que estão concorrendo a uma mesmo recurso. Caso possuam a mesma prioridade, a que chegou primeiro a fila de escalonamento levará a melhor. Caso as duas *threads* possuam prioridades distintas, a de maior valor levará a melhor. A macro que define esse comportamento por parte do escalonador é *SCHED_FIFO*; na segunda política, *Round-Robin*, as *threads* de igual prioridade serão escalonadas segundo o algoritmo de mesmo nome, podendo algumas delas sofrer por preempções. A macro utilizada para esse comportamento é a *SCHED_RR*; Existe também uma terceira política, definida pela macro *SCHED_OTHER*, usada para sinalizar que as *threads* não mais precisam de uma política específica de escalonamento.

O mais simples de se compreender, a herança de escalonamento, define para a *thread* que será criada se ela deve herdar os parâmetros e as políticas de escalonamento de sua *thread* criadora ou se deve obedecer aos valores presentes na variável atributo. A macro `PTHREAD_INHERIT_SCHED` diz a *thread* recém criada que use os parâmetros e a política de escalonamento de sua *thread* criadora. Ou seja, ignora qualquer chamada as `pthread_attr_setschedparam` e `pthread_attr_setschedpolicy`; Por outro lado, a macro `PTHREAD_EXPLICIT_SCHED` define que a nova *thread* deverá utilizar os valores presentes na variável atributo.

2.3. Sincronização

O modelo de paralelismo de usado pelas *threads* é comumente chamado de *múltiplos processadores de memória compartilhada*. Em outras palavras, toda a região de memória é acessível por qualquer *thread*. Apesar de trazer vantagens quanto ao acesso dos dados por diversas *threads* pois evita o uso de mecanismo de comunicação entre processos, essa liberdade do acesso a memória pode trazer, quando má utilizados, diversos problemas um tanto quanto difíceis de diagnosticar. A maioria desses problemas são ocasionados pela não atomicidade das operações realizadas nos fluxos de processamento.

Como um exemplo simples podemos usar o acesso concorrente a uma estrutura de dados dinâmica (aqui será uma pilha). O código segue abaixo:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define TRUE      1
#define FALSE     0

#define MAX_THREADS 10

typedef struct node {
    void *valor;
    struct node *proximo;
} No;

typedef struct stack {
    No *no;
    int qtde;
} Pilha;

Pilha *pilha;

Pilha* NovaPilha () {
    Pilha *nova = (Pilha *) calloc (1, sizeof(Pilha));

    if (nova) {
        return nova;
    }
}
```

```

        return NULL;
    }

    int Empilhar (Pilha *p, void *valor) {
        No *no;

        if (!p)
            return FALSE;

        no = (No *) calloc (1, sizeof(No));

        if (no) {
            no->proximo = p->no;
            p->no = no;

            no->valor = valor;

            p->qtde++;

            return TRUE;
        }

        return FALSE;
    }

    No* Desempilhar (Pilha *p) {
        if (!p)
            return FALSE;

        if (p->no) {
            No *no = p->no;
            p->no = no->proximo;

            p->qtde--;

            return no;
        }

        return NULL;
    }

    void *threadfa (void *data) {
        long valor;

        while (TRUE) {

```

```

        valor = random ();
        fprintf (stderr, "Empilhando %ld\n", valor);
        Empilhar (pilha, (void *) valor);
    }

    return NULL;
}

void *threadfb (void *data) {
    No *no;
    int valor;

    while (TRUE) {
        no = Desempilhar (pilha);

        if (no) {
            fprintf (stderr, "Desempilhando %ld\n", (long) no->valor);
            free (no);
        }
    }

    return NULL;
}

int main () {
    pthread_t threads[MAX_THREADS];
    int i;

    pilha = NovaPilha();

    for (i = 0 ; i < MAX_THREADS / 2 ; i++)
        pthread_create (&threads[i], NULL, threadfa, NULL);

    for ( ; i < MAX_THREADS ; i++)
        pthread_create (&threads[i], NULL, threadfb, NULL);

    pthread_join (threads[0], NULL);
}

```

Apesar de correto do ponto de vista sequencial, em ambientes *multithreads* o nosso código apresentará no mínimo três erros: *Segmentation Fault*, *Double Free* e alocação excessiva de memória (*Aborted*).

O terceiro erro não nos interessa muito, mas vale citar sua causa: caso as decisões de escalonamento sejam tais que as fluxos de execução que tenham como função *threadfa* rodem muito mais vezes que os outros fluxos, o núcleo julgará que a aplicação está fazendo mal uso dos recursos de memória e a abortará.

Também causados pelas decisões de escalonamento, os dois primeiros erros podem ser evitados com o uso de mecanismos de controle de concorrência. Porém, antes de conhecer tais mecanismos, vale primeiramente entender a causa dos erros.

Pelo código podemos ver que teremos $MAX_THREADS/2$ fluxos executando a função de empilhamento, *threadfa*; e $MAX_THREADS/2$ fluxos executando a função de desempilhamento e liberando os dados, *threadfb*.

Consideremos o programa executando. Digamos que já tenhamos alguns poucos valores empilhados. Agora, consideremos que dois fluxos que estejam desempilhando executem e sejam interrompidos exatamente na quinta linha da função *Desempilhar* (*No *no = p->no;*). Agora digamos que um desses fluxos volte ao processador e seja executado até a nona linha da função *threadfb* (*free (no);*). Ou seja, depois do retorno da função *Desempilhar* e da liberação do endereço presente em *no*.

No cenário descrito acima, podemos ver claramente que o fluxo de execução que não avançou estará trabalhando com um valor de memória inconsistente. Portanto, sua próxima instrução (*p->no = no->proximo*) acarretaria num *Segmentation Fault*.

Agora, analisemos um segundo cenário. Ainda tendo alguns poucos valores empilhados, consideremos que dois fluxos, mais uma vez desempilhando, executem e sejam interrompidos na quinta linha da função *Desempilhar*. Agora, digamos que os dois fluxos avancem até que a função retorne. Nesse cenário, ambos os fluxos liberarão o endereço presente em *no*, o que ocasionará um *Double Free* e a execução será finalizada.

Um último, e bem elaborado caso, seria no uso de uma biblioteca de *Garbage Collection* (GC). Essa bibliotecas permitem ao desenvolvedor uma maior flexibilidade na dealocação de memória, tornando desnecessário o uso de funções como o *free*. Portanto, a própria biblioteca se encarregaria de liberar os dados. Logo, não existiria a oportunidade para um *Double Free*. Entretanto, teríamos um quarto erro: inconsistência dos dados. Digamos que *n* fluxos de desempilhamento avancem até a quinta linha da função *Desempilhar*. Agora, digamos que antes da execução de cada um desses fluxos, um fluxo de empilhamento seja executado. Cada um dos valores recém empilhados seriam perdidos pois na próxima instrução dos fluxos de desempilhamento o endereço do topo da pilha seria ajustado para outro segmento de memória. Ou seja, os *n* novos valores empilhados seriam colhidos pelo GC.

Aos trechos de códigos acessados e modificados concorrentemente, denominamos *seções críticas*. Em outras palavras, todo código que necessita do uso de mecanismo de sincronização para garantir sua integridade.

Alguns podem argumentar que diversas verificações resolveriam os problemas. Porém, já é matematicamente provado que, dado um conjunto de instruções que não utilizam diretivas de sincronização e que realizam acessos concorrentes a segmentos de memória, existe um conjunto de decisões de escalonamento que levam o sistema a um estado inconsistente.

2.3.1. Variáveis de Exclusão Mútua

Como sugerido pelo nome, essas variáveis garantem que apenas um dos fluxos de processamento tenham acesso a uma seção crítica. Elas realizam seu trabalho usando uma técnica de bloqueio com espera ociosa. Para tal comportamento é necessário o uso de duas funções, *lock* e *unlock*.

Antes de entrar numa seção crítica, o fluxo corrente precisa pedir passagem a variável de exclusão mútua através da função *lock*. Caso nenhum outro fluxo esteja nessa seção crítica, o fluxo corrente ganha passagem. Esse processo é comumente chamado de "Aquisição" da variável; Caso outro fluxo esteja na vez, o fluxo corrente permanece bloqueado até que seja chamada a função *unlock* na variável de exclusão mútua em questão. Para casos onde é preferível fazer outro processamento ao invés de esperar na variável de exclusão mútua, existe a função *trylock*. Ela testa primeiramente a variável, e caso já esteja bloqueada, ela sinaliza ao fluxo através de seu código de retorno. Caso contrário, o fluxo entra na seção crítica. Para sinalizar a saída da seção crítica, o fluxo de processamento deve chamar a função *unlock*, num processo chamado de "liberação" da variável.

No POSIX threads, o tipo das variáveis de exclusão mútua são chamados de *pthread_mutex_t*. As funções de *lock*, *unlock* e *trylock* são chamadas de *pthread_mutex_lock*, *pthread_mutex_unlock* e *pthread_mutex_trylock*. Apartir de agora abreviaremos "variáveis de exclusão mútua" para "mutex" (do inglês, *MUTual EXclusion*). Antes de utilizar uma *mutex* é necessário inicializá-la com a função *pthread_mutex_init*.

Como dito pelo cientista computacional Edsongley, não existe documentação melhor que código fonte, vamos a um rápido e abstrato exemplo de uso de *mutexes*:

```
/* Em um dos fluxos */
pthread_mutex_t mutex;
pthread_mutex_init (&mutex, NULL);

/* Nos fluxos que necessitam de sincronização */
pthread_mutex_lock (&mutex);
/* seção crítica */
pthread_mutex_unlock (&mutex);
```

Um exemplo, mais uma vez abstrato, de uso do *trylock* seria:

```
if (!pthread_mutex_trylock (&mutex)) {
    /* seção crítica */
    pthread_mutex_unlock (&mutex);
} else {
    /* processamento alternativo */
}
```

Estando o funcionamento básico explicado, vamos agora ao nosso código de acesso concorrente a pilha.

Primeiramente precisamos verificar quais pontos realmente precisam de sincronização. A maneira mais simples e rápida seria sincronizar a chamada das funções

Empilhar e Desempilhar, bastando apenas inserir uma *mutex* global e adicionar as chamadas a *pthread_mutex_lock* e *pthread_mutex_unlock* nas funções *threadfa* e *threadfb*. Ficaria mais ou menos assim:

```
/* ... */

#define MAX_THREADS 20

pthread_mutex_t mutex;

/* ... */

void *threadfa (void *data) {
    long valor;

    while (TRUE) {
        valor = random ();
        fprintf (stderr, "Empilhando %ld\n", valor);

        pthread_mutex_lock (&mutex);
        Empilhar (pilha, (void *) valor);
        pthread_mutex_unlock (&mutex);
    }

    return NULL;
}

void *threadfb (void *data) {
    No *no;
    int valor;

    while (TRUE) {
        pthread_mutex_lock (&mutex);
        no = Desempilhar (pilha);
        pthread_mutex_unlock (&mutex);

        if (no) {
            fprintf (stderr, "Desempilhando %ld\n", (long) no->valor);
            free (no);
        }
    }

    return NULL;
}

int main () {
    pthread_t threads[MAX_THREADS];
```

```

int i;

pilha = NovaPilha();

pthread_mutex_init (&mutex, NULL);

for (i = 0 ; i < MAX_THREADS / 2 ; i++)
    pthread_create (&threads[i], NULL, threadfa, NULL);

for ( ; i < MAX_THREADS ; i++)
    pthread_create (&threads[i], NULL, threadfb, NULL);

pthread_join (threads[0], NULL);
}

```

Essa é conhecida como o método "inocente" (do inglês, *naive method*). Ela funciona, porém não é a mais inteligente (eficiente).

Analisando-se bem o código podemos ver que existe um grande desperdício de tempo usando-se o método inocente. Primeiro, impedimos a chamada da função: a empilhagem dos argumentos e do endereço de retorno também estará sendo sincronizado, o que é totalmente desnecessário!; Segundo, temos uma alocação dinâmica ocorrendo na função *Empilhar*. Alocação dinâmica exige uma troca de contexto para sua execução. Trocas de contexto devem ser evitadas ao máximo;

Portanto, seria mais eficiente o controle de concorrência ser inerente a estrutura de dado *Pilha*. Ou seja, a própria estrutura de dados teria uma *mutex* e o *lock* e *unlock* seriam feitos pelas funções *Empilhar* e *Desempilhar*. Além de mais eficiente, podemos dizer que é mais seguro. Imagine uma novo programador entrando na equipe de desenvolvimento e esquecendo que deveria sincronizar as chamadas? Então, nosso código ficaria como exposto abaixo:

```

typedef struct stack {
    No *no;
    int qtde;

    pthread_mutex_t mutex;
} Pilha;

Pilha *pilha;

Pilha* NovaPilha () {
    Pilha *nova = (Pilha *) calloc (1, sizeof(Pilha));

    if (nova) {
        pthread_mutex_init (&nova->mutex, NULL);
        return nova;
    }
}

```

```

        return NULL;
    }

    int Empilhar (Pilha *p, void *valor) {
        No *no;

        if (!p)
            return FALSE;

        no = (No *) calloc (1, sizeof(No));

        if (no) {
            pthread_mutex_lock (&p->mutex);

            no->proximo = p->no;
            p->no = no;

            no->valor = valor;

            p->qtde++;

            pthread_mutex_unlock (&p->mutex);

            return TRUE;
        }

        return FALSE;
    }

    No* Desempilhar (Pilha *p) {
        if (!p)
            return FALSE;

        pthread_mutex_lock (&p->mutex);

        if (p->no) {
            No *no = p->no;
            p->no = no->proximo;

            p->qtde--;

            pthread_mutex_unlock (&p->mutex);

            return no;
        }
    }

```



```

pthread_mutex_unlock (&p->mutex);

return NULL;
}

```

Como é de costume encontrar por aí, temos uma estrutura de dados do tipo *fila thread-safe*.

Deve-se observar que até o acesso a variável *Pilha->no* na instrução de controle de fluxo *if* da função *Desempilhar* foi sincronizado. Lembrem-se que podemos encontrar um conjunto de decisões de escalonamento que possa fazer com que o valor lido esteja inconsistente logo após a verificação.

Atributos das variáveis de exclusão mútua

Como devem ter notado, a função *pthread_mutex_init* recebe dois parâmetros. O primeiro é o endereço da *mutex* que desejamos inicializar e o segundo, até agora, só apareceu como nulo. Esse último serve para que o comportamento das *mutex* seja alterado conforme a necessidade. Para tal objetivo temos o tipo *pthread_mutexattr_t*. Esse tipo seria equivalente ao *pthread_attr_t*, só que aplicado as *mutexes*.

Para usarmos tais variáveis, devemos inicializá-las com a função *pthread_mutexattr_init*. Para destruímos, usamos a função *pthread_mutexattr_destroy*. Para utilizá-las, basta passar seus endereços como segundo parâmetro da função *pthread_mutex_init*. Abstratamente:

```

pthread_mutexattr_t mutexattr;
pthread_mutex_t mutex;

/* ... */

pthread_mutexattr_init (&mutexattr);
pthread_mutex_init (&mutex);

/* Definição de alguns comportamentos especiais... */

pthread_mutex_init (&mutex, &mutexattr);

/* ... */

pthread_mutexattr_destroy(&mutexattr);

```

Veremos agora os comportamentos mais interessantes associados com as variáveis atributos das *mutexes*.

Escopo da variável de exclusão mútua

Assim com as *threads*, as *mutexes* podem existir em dois contextos diferentes: processo ou sistema. No escopo do processo, a *mutex* reside num espaço de endereçamento local e somente o processo que criou a *mutex* pode utilizá-la. Já no escopo do sistema, a *mutex* pode ser posta num espaço de memória compartilhado e ser utilizada por diversos processo.

Observe que o escopo de uma *mutex* só está relacionado com o espaço de endereçamento que a mesma reside. Ou seja, para que uma *mutex* seja usada por diversos fluxos de escopo de sistema criadas por um mesmo processo, ela não necessita estar no escopo do sistema!!!

A função usada para determinar o escopo da *mutex* é *pthread_mutexattr_setpshared* (*pthread_mutexattr_t**, *int*). O primeiro parâmetro é a variável atributo; o segundo é o escopo da *mutex*. *PTHREAD_PROCESS_SHARED* tornará possível o uso da *mutex* por diversos processos. Já *PTHREAD_PROCESS_PRIVATE* impedirá qualquer tentativa de uso da *mutex* por outro processo. Vale enfatizar que comportamentos arbitrários ocorrerão caso se tente usar uma *mutex* privada entre diversos processos. Por padrão as *mutex* residem num escopo local.

Explicar como utilizar uma *mutex* compartilhada entre processos foge do tema desse artigo, mas quem quiser saber mais um pouco e ainda ver uma simples e claro exemplo pode dar uma olhada nas páginas de manual de (3p) *pthread_mutexattr_init*.

Tipo da variável de exclusão mútua

Para entender o por que de se ter diversos tipos de *mutexes*, faz-se necessário entender o conceito de *deadlock*.

Deadlocks, ou empasses fatais, são situações onde o progresso do fluxo de execução de um processo será interrompido devido ao mesmo permanecer aguardando um evento que nunca ocorrerá.

Um exemplo simples de *deadlock* são observados com o uso de *mutexes*. Um questionamento simples: o que ocorre quando um mesmo fluxo de processamento tenta adquirir o controle de uma *mutex* duas vezes consecutivas sem a liberar? *Deadlock!* Ou seja, o fluxo será interrompido até que o fluxo que está com o controle da *mutex* a libere. Mas o fluxo interrompido é o fluxo que deve liberar a *mutex*! Uma espécie de paradoxo.

Para evitar problemas relacionados com empasses, foram propostos diversos tipos de *mutexes*. A saber:

Rápidas (*fast mutex*): As mais simples. Uma segunda tentativa de aquisição numa *mutex* desse tipo causará uma situação de inanição (o fluxo permanecerá eternamente bloqueado);

Recursivas (*recursive mutex*): Sendo um pouco mais sofisticadas, essas *mutexes* lembram quantas vezes foram adquiridas por um mesmo fluxo. O fluxo deve liberá-las a mesma quantidade de vezes para que outro fluxo possa adquiri-lá.

Checagem de Erro (*error checking mutex*): Uma segunda tentativa de adquirir essa *mutex* retorna o código de erro EDEADLK;

Muitas vezes fica a dúvida: se temos como evitar *deadlocks* com o uso de *mutex* recursivas ou com checagem de erros, por que usar uma *mutex* rápida se isso tornará o processo de descoberta de erro mais difícil? Uma das respostas seria eficiência. Os tipos mais sofisticados de *mutexes* nada mais são que uma extensão do tipo rápido. Por exemplo, caso não tenhamos uma implementação de *PThreads* com suporte a recursão, podemos fazê-lo da seguinte forma:

```
typedef struct rmutex {
```

```

    pthread_mutex_t check;
    pthread_mutex_t lock;
    pthread_t id;

    int count;
} RMutex;

int RMutexLock (RMutex* mutex) {
    pthread_mutex_lock (&mutex->check);

    if (mutex->id == pthread_self()) {
        mutex->count++;
        pthread_mutex_unlock (&mutex->check);

        return 0;
    }

    pthread_mutex_unlock (&mutex->check);

    pthread_mutex_lock (&mutex->lock);
    pthread_mutex_lock (&mutex->check);

    mutex->id = pthread_self ();
    mutex->count = 1;

    pthread_mutex_unlock (&mutex->check);

    return 0;
}

int RMutexUnlock (RMutex* mutex) {
    pthread_mutex_lock (mutex->check);

    if (mutex->id == pthread_self()) {
        if (--mutex->count) {
            mutex->id = NULL;
            pthread_mutex_unlock (&mutex->lock);
        }

        pthread_mutex_unlock (&mutex->check);

        return 0;
    }

    pthread_mutex_unlock (&mutex->check);

```

```
    return 1;
}
```

Não entraremos na discussão do código, serve apenas para mostrar que o que foi dito acima.

Vejam que foi necessário manter um contador, uma identificação de qual *thread* adquiriu o *mutex* e ainda uma *mutex* extra para que as verificações básicas possam ser feitas. As *mutexes* com checagem de erro chegam até a ter uma implementação mais simples.

Enfatizando, essa não é a implementação das *mutexes* recursivas nativa do *PThread*, é apenas uma ilustração que as *mutexes* mais sofisticadas podem derivar das *mutexes* rápidas.

Enquanto uma *mutex* com checagem de erro provem apenas um mecanismo simplório que não altera muito o comportamento das *mutexes* rápidas, as *mutexes* recursivas trazem consigo algumas complicações. No código de exemplo, podemos ver que a *mutex lock* só será liberada quando a variável *count* for igual a zero. Caso não hajam liberações suficientes, o sistema pode mais uma vez sofrer por inanição.

2.3.2. Variáveis Condicionais

Digamos que temos uma determinada região de código que a *thread* só deverá executar após uma determinada condição se tornar verdadeira. Digamos também que essa condição será estabelecida por uma outra *thread* e que a verificação da mesma envolva o acesso a uma região crítica. Utilizando uma *mutex*, poderíamos fazer:

```
while (1) {
    pthread_mutex_lock (&mutex);

    if (condicao) {
        /* faz alguma coisa */
        pthread_mutex_unlock (&mutex);

        return 1;
    }

    pthread_mutex_unlock (&mutex);
}
```

Apesar de funcionalmente correto, o código possui um problema simples: carece de espera ociosa. Ou seja, as *threads* que aguardarão a manutenção da condição irão permanecer executando as instruções presentes no *loop*, evitando que o processador seja alocado para alguma outra *thread*. É justamente aqui que as variáveis condicionais entram.

As variáveis condicionais são um mecanismo um pouco mais poderoso para a sincronização das *threads*. Permitem especificar situações mais complexas acerca da execução das *threads*, além de garantir que a espera seja feita no ócio.

Todavia, as variáveis condicionais estão sempre acompanhadas de *mutexes*. O motivo é a garantia de um predicado que deverá manter-se durante toda a execução. Em outras palavras, como no código acima sempre haverá uma região crítica.

O tipo associado com as variáveis condicionais são o *pthread_cond_t*. Antes de utilizá-los, como de praxe, é necessária sua inicialização em meio ao uso da função *pthread_cond_init* (*pthread_cond_t **, *pthread_condattr_t **) ou atribuir o valor *PTHREAD_COND_INITIALIZER*. A primeira forma de inicialização se mostra necessária apenas quando deseja-se modificar o comportamento da variável em meio ao uso de uma variável atributo. Caso deseje apenas uma variável condicional padrão, a atribuição é mais eficiente.

Após inicializada, as funções *pthread_cond_wait* (*pthread_cond_t **, *pthread_mutex_t **) e *pthread_cond_timedwait* (*pthread_cond_t **, *pthread_mutex_t **, *struct timespec **) podem ser usadas para aguardar a condição. Os primeiros argumentos de ambas são idênticos: uma variável condicional já inicializada e uma *mutex* inicializada adquirida pela *thread* atual. A função *pthread_cond_timedwait* irá bloquear somente enquanto a hora do sistema for menor que o valor especificado pelo seu terceiro argumento.² Quando chamadas, as funções *pthread_cond_wait* e *pthread_cond_timedwait*, liberam a *mutex* passada como segundo argumento.

Para sinalizar as *threads* que estão bloqueadas na variável condicional são providas duas funções: *pthread_cond_signal* (*pthread_cond_t **) e *pthread_cond_broadcast* (*pthread_cond_t **). O primeiro argumento de ambas as funções é a variável argumento a qual se deseja sinalizar. As duas possuem uma diferença semântica bastante sucinta: *pthread_cond_signal* irá desbloquear uma única *thread*, aleatoriamente mas levando em consideração as prioridades associadas a cada *thread*; por sua vez, *pthread_cond_broadcast* irá desbloquear todas as *threads* que estiverem aguardando pelo sinal.

Então, o código exemplo dessa sessão pode ser re-escrito da seguinte maneira:

```
pthread_mutex_lock (&mutex);

if (!condicao) {
    pthread_cond_wait (&cond, &mutex);
}

if (condicao) {
    /* Faz algo... */
    pthread_mutex_unlock (&mutex);
} else {
    pthread_mutex_unlock (&mutex);
    return -1;
}
```

Esse fragmento possui a mesma semântica que o anterior, porém, devido ao uso das variáveis condicionais, não é necessário a permanência em um *looping* infinito nem

²Para um melhor detalhamento do tipo *struct timespec* veja <http://opengroup.org/onlinepubs/007908799/xsh/time.h.html>

muito menos realizar constantes aquisições e liberações da *mutex*.

Analisando o código, primeiramente vemos que é feita a aquisição da *mutex* pela *thread*; com a *mutex* em mãos, a *thread* verifica logo em seguida se a variável *condicao* possui valor diferente de zero. Caso o valor da variável *condicao* seja zero, *pthread_cond_wait* irá por a *thread* para escutar na variável condicional *cond* e liberará a *mutex* para que a *thread* emissora do sinal de execução possa modificar os dados. Caso o valor seja diferente de zero ou após o retorno de *pthread_cond_wait*, o código do bloco de execução irá alterar os dados compartilhados entre as *threads* (*/* Faz algo... */*) e logo em seguida liberará a *mutex*;

As boas práticas (e o manual) dizem que mesmo recebendo a sinalização de uma outra *thread*, deve-se re-avaliar os predicados envolvidos. No nosso caso, mais uma comparação foi realizada na variável *condicao*.

A *thread* emissora do sinal de execução possuirá um código mais simples, que no nosso exemplo seria algo como o fragmento a seguir:

```
pthread_mutex_lock (&mutex);  
condicao = facaalgo();  
pthread_cond_signal (&cond);  
pthread_mutex_unlock (&mutex);
```

Ou seja, o emissor precisará adquirir a *mutex*; alterar os dados compartilhados de forma a garantir a condição de execução, que em nosso exemplo é apenas a variável *condicao*; sinalizar para as *threads* que estão aguardando o sinal; e liberar a *mutex*.

O exemplo mostrado é simples, mas com um pouco de imaginação é possível criar diversas condições avançadas, podendo até "imitar" o comportamento de diversos outros mecanismos de sincronização. Os semáforos, por exemplo, são muito simples de implementar através de *mutexes* e variáveis condicionais. Será apresentado o comportamento dos semáforos a seguir, portanto, quem quiser tentar implementá-los, fica como tarefa de casa, ehehehe...

2.3.3. Semáforos

Os semáforos trabalham como sinalizadores: seu principal uso é para representar a disponibilidade de um recurso computacional. Por exemplo, seja uma fila dinâmica controlada por uma *mutex* e duas *threads*, uma consumidora dos elementos da fila e outra produtora desses elementos. Uma das maneiras de garantir a espera ociosa nesse situação seria criar uma variável condicional e usar o contador de elementos da fila como condição de execução. Simples e fácil. Mas como essa é uma situação recorrente nos programas paralelos/*multithread*, resolveu-se criar uma estrutura de dados para tratá-la: os semáforos.

Essas estruturas de dados possuem duas funções; *post* e *wait*. A *wait* faz com que a *thread* bloqueie até que outra *thread* sinalize no semáforo, tarefa feita através da função *post*. Em outras palavras: *wait* espera e *post* dá a largada.

O primeiro passo para utilizar os semáforos é incluir o arquivo de cabeçalho *semaphore.h*. As funções *wait* e *post*, na biblioteca *PThreads*, estão implementadas como *int sem_wait(sem_t *sem)* e *int sem_post(sem_t *sem)*, respectivamente. Existem também

as funções *sem_trywait*, que verifica se o contador do semáforo está positivo e caso esteja decrementa-o mas não bloqueia caso o contador esteja em zero; e *sem_timedwait*, que permite especificar um intervalo de tempo no qual a *thread* pode ficar bloqueada. Para maiores informações sobre ambas, o manual *on-line* tem uma ótima explicação.

Antes de utilizar um semáforo, precisamos inicializá-lo. A função responsável por essa tarefa é a *int sem_init(sem_t *, int, unsigned int)*. O primeiro argumento é o semáforo a ser inicializado, o segundo é uma *flag* para indicar se o semáforo deverá ser compartilhado entre múltiplas *threads* ou múltiplos processos. O valor 0 (zero) indica que o semáforo será utilizado apenas pelas *threads* de um mesmo processo; o último argumento é o valor inicial do contador do semáforo. É um erro horrível utilizar um semáforo selvagem e é bem difícil de visualizar o erro. Tomem cuidado com essa parte.

Não é necessário um exemplo complexo para entender o uso dos semáforos. Então, vai um bem abstrato e simples.

```
/* Nosso semáforo */
sem_t semaforo;

/*
    Semáforos, em sua maioria, são acompanhados
    de mutexes.
*/
pthread_t mutex;

/*
    Semáforo compartilhado entre as threads de
    um mesmo processo e com o contador vazio.
*/
sem_init (&semaforo, 0, 0);
pthread_mutex_init (&mutex, NULL);

/* Excerto da Thread Produtora */
while (1) {
    pthread_mutex_lock(&mutex);
    /* Função de produção */
    produzir();
    pthread_mutex_unlock (&mutex);

    sem_post(&semaforo);
}

/* Excerto da Thread Consumidora */
while (1) {
    sem_wait(&semaforo);

    pthread_mutex_lock(&mutex);
    /* Função de consumo */
```

```

    consumir();
    pthread_mutex_unlock (&mutex);
}

```

Bem simples: a thread consumidora vai ficar esperando no semáforo semaforo e quando a thread produtora sinalizar nessa variável, a thread consumidora será desbloqueada. Antes de produzir e antes de consumir, as threads entram numa sessão crítica, usando a variável de exclusão mútua mutex para impedir inconsistências.

3. Problemas Clássicos de Sincronização

Visando ilustrar melhor os problemas enfrentados pelos desenvolvedores relativos a sincronização entre threads/processos, os diversos pesquisadores da área criaram problemas no mínimo peculiares. Vou comentar e mostrar a solução de dois deles: o **jantar dos filósofos** e o **barbeiro dorminhoco**.

3.1. O Jantar dos Filósofos

Esse problema foi proposto em 1965 por Edsger Dijkstra, uma das maiores mentes da computação. Ele baseia-se na generalização de que os filósofos só fazem duas coisas na vida: comer e pensar. Mas como nem tudo na vida é tão fácil, o jantar dos filósofos consiste num suculento e escorregadio espagheti, tanto que, para comer o espagheti, são necessários dois garfos!

Aí vem a pergunta: como isso pode ser modelado num problema de sincronização? De forma bem simples. Considere que estão sentados a mesa N filósofos e que estão disponíveis N garfos. Considerando que não vale usar as mãos, poderemos ter no máximo $N/2$ filósofos comendo ao mesmo tempo. Ainda não sacou por que esse é um problema de sincronização? Imagine se cada filósofo, ao mesmo tempo, pegar o garfo a sua direita e não soltá-lo enquanto aguarda a disponibilidade do garfo a esquerda. Ocorrerá o que chamamos de *deadlock*: cada um dos filósofos segurará um garfo e não irá soltá-lo até que o filósofo a sua esquerda disponibilize outro garfo. Só que nenhum outro filósofo soltará seu garfo! Portanto, todos os filósofos morrerão por falta de comida, ou o termo técnico advindo da biologia, inanição (*starvation* em inglês). Não é necessário dizer que o contrário, pegar os garfos a esquerda e esperar o da direita, também leva a uma situação problemática.

Existem diversas abordagens que resolve esse problema. Irei focar-me aqui numa mais simples, que apesar de funcional, foje um pouco da solução ideal. O código está abaixo.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include <pthread.h>

#define ESQUERDA(f) ((f + nfilosofo - 1) % nfilosofo)
#define DIREITA(f) ((f + 1) % nfilosofo)

```



```

#define MSLEEP 5
#define SLEEPTIME (random()%MSLEEP)

typedef enum {
    COMFOME, PENSANDO, COMENDO,
} Filosofo;

pthread_mutex_t garfos;
Filosofo *filosofos;

int nfilosofo;

void *ffuncao(void *f) {
    int fid = (int) f;

    while (1) {
        pthread_mutex_lock (&garfos);

        switch (filosofos[fid]) {
            case PENSANDO:
                filosofos[fid] = COMFOME;
                pthread_mutex_unlock(&garfos);

                fprintf(stdout, "F[%d]: Estou pensando...\n",
                    fid);

                sleep(SLEEPTIME);
                break;

            case COMFOME:
                fprintf(stdout, "F[%d]: Estou com fome...
                    Vou tentar pegar os garfos!\n", fid);

                if (filosofos[ESQUERDA(fid)] == COMENDO) {
                    pthread_mutex_unlock(&garfos);
                    fprintf (stdout, "\tFilosofo %d estah
                        comendo... Nao deu pra mim...
                        :-( \n", ESQUERDA(fid));
                } else if (filosofos[DIREITA(fid)] == COMENDO) {
                    pthread_mutex_unlock(&garfos);
                    fprintf (stdout, "\tFilosofo %d estah
                        comendo... Nao deu pra mim...
                        :-( \n", DIREITA(fid));
                } else {
                    filosofos[fid] = COMENDO;
                    pthread_mutex_unlock(&garfos);
                }
            }
        }
    }
}

```

```

        fprintf (stdout, "\tAeee! Vou encher o
                    bucho!  :-)\n");
    }

    sleep(SLEEPTIME);
    break;

case COMENDO:
    filosofos[fid] = PENSANDO;
    pthread_mutex_unlock(&garfos);

    fprintf(stdout, "F[%d]: Enchi o bucho... Hora de
                    voltar a pensar...\n", fid);

    sleep(SLEEPTIME);
    break;
    }
}

int main (int argc, char **argv) {
    int i;
    pthread_t *t;

    if (argc > 1) {
        char **endptr = NULL;

        nfilosofo = (int) strtol(argv[1], endptr, 10);

        if (endptr) {
            fprintf (stderr, "Argumento inválido: %s\n", argv[1]);
            return 1;
        }
    } else {
        nfilosofo = 5;
    }

    filosofos = (Filosofo *) calloc (nfilosofo, sizeof(Filosofo));

    if (!filosofos) {
        fprintf(stderr, "OS filósofos estão cansados.
                    Não querem pensar hoje...\n");
        return 1;
    }

    t = (pthread_t *) calloc (nfilosofo, sizeof(pthread_t));

```

```

if (!t) {
    fprintf (stderr, "Executar ou não executar ...
                                ... eis a questão!\n");
    return 1;
}

pthread_mutex_init (&garfos, NULL);

for (i = 0 ; i < nfilosofo ; i++) {
    filosofos[i] = PENSANDO;
    pthread_create (&t[i], NULL, ffuncao, (void *) i);
}

for (i = 0 ; i < nfilosofo ; i++) {
    pthread_join(t[i], NULL);
}

return 0;
}

```

A principal função do nosso código é a *ffuncao*. Ela é a função encarregada por manter consistente os estados dos filósofos, que por sua vez, estão representados pela *enum filosofo*. De acordo com o código, podemos ver que os filósofos poderão estar em três estados distintos: COMFOME, PENSANDO e COMENDO.

No início da execução, todos os filósofos são postos no estado PENSANDO, como visto na linha 101. Ao iniciar a função *ffuncao*, os filósofos são postos no estado de FOME. Quando nesse estado, o filósofo que está executando observa o filósofo a sua esquerda e em seguida o filósofo a direita. Caso ambos estejam PENSANDO ou COMFOME, o filósofo atual entra no estado de COMENDO. Caso contrário, o filósofo continua no estado de COMFOME e espera um tempo arbitrário para tentar novamente. Observe que a variável *filosofo*, lida e modificada concorrentemente por todas as *threads*, está protegida pelo *mutex garfos*.

Notem também que é importante liberar a *mutex* tão logo o trabalho necessário seja realizado. Por exemplo, no estado COMFOME, a *mutex* é liberada tão logo o próximo estado do filósofo possa ser inferido.

O Andrew Tanenbaum, famoso por criar o *Minix*, apresenta em seu livro *Sistemas Operacionais Modernos* uma solução bem mais sofisticada, utilizando espera ociosa e tudo o mais. Recomendo a todos dar uma olhada. A Wikipédia, em inglês, também tem algumas soluções diferentes da mostrada aqui. Recomendo-a também.

3.2. O Barbeiro Dorminhoco

Atribuído também ao Edsger Dijkstra, o problema do barbeiro dorminhoco também apresenta uma inusitada analogia. Dessa vez, o problema envolve um barbeiro que sempre aproveita o tempo livre para tirar uma soneca, mas não se importa por ser acordado pelos clientes.

Nossa missão nesse problema é garantir que tão logo quanto chegue, o cliente seja atendido pelo barbeiro. Caso o barbeiro esteja ocupado, os novos clientes se sentarão nas cadeiras de espera até que sejam atendidos. Se não houver nenhuma cadeira disponível, os novos clientes vão embora. Se não houver clientes, o barbeiro vai tirar uma soneca.

O problema mais simples, onde existe apenas um barbeiro, é conhecido também como o problema do barbeiro dorminhoco simples. Nossa implementação será baseada numa versão mais sofisticada, com N barbeiros, que apesar de tudo a solução é tão fácil quanto a do problema simples. Os clientes chegarão a cada 1 segundo, com $P\%$ de probabilidade de entrarem na barbearia. O código está abaixo.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>

#include <semaphore.h>

int  nlugares;
int  nbarbeiros;
int  probabilidade;

int      dlugares;
sem_t    slugares;
pthread_mutex_t  mlugares;

void *fbarbeiro (void *b) {
    int bid = (int) b;
    int q;

    while (1) {
        if (sem_trywait (&slugares) == -1) {
            if (errno != EAGAIN) {
                perror("Erro");
                break;
            }
        }

        fprintf (stdout, "Barbeiro[%d]: Não tem nenhum
                        cliente...\n\tVou dormir...
                        zZzzZzZ...\n\n", bid);

        sem_wait (&slugares);
    }

    pthread_mutex_lock (&mlugares);
    q = --dlugares;
```

```

        pthread_mutex_unlock (&mlugares);

        fprintf (stdout, "Barbeiro[%d]: Atendendo
                        cliente...\n\t Ainda existem %d
                        clientes na fila.\n\n", bid, q);

        sleep (random() % 5);
    }

    return NULL;
}

void gerarclientes() {
    int q;
    int lugardiponivel;

    sleep (2);

    while (1) {
        if ((random() % 100) < probabilidade) {
            pthread_mutex_lock (&mlugares);

            if (dlugares < nlugares)
                lugardiponivel = 1;
            else
                lugardiponivel = 0;

            if (lugardiponivel) {
                q = ++dlugares;
                sem_post(&slugares);
            }

            pthread_mutex_unlock (&mlugares);

            if (lugardiponivel) {
                fprintf (stdout, "Um novo cliente chegou...\n\t
                                Ele está aguardando sua vez.\n\n");
            } else {
                fprintf (stdout, "Um novo cliente chegou...\n\t
                                O salão está lotado. Ele virá outro
                                dia.\n\n");
            }

            fprintf (stdout, "Existem %d clientes na fila e
                            %d lugares disponíveis.\n\n",
                            q, nlugares - q);

```

```

        }

        sleep(1);
    }
}

int bloquearsinais () {
    sigset_t set;

    sigemptyset(&set);

    sigaddset(&set, SIGINT);
    sigaddset(&set, SIGQUIT);
    sigaddset(&set, SIGTERM);

    if (pthread_sigmask(SIG_BLOCK, &set, NULL)) {
        fprintf(stderr, "Não foi possível mascarar os sinais!");
        return 1;
    }

    return 0;
}

int liberarsinais () {
    sigset_t set;

    sigemptyset(&set);

    sigaddset(&set, SIGINT);
    sigaddset(&set, SIGQUIT);
    sigaddset(&set, SIGTERM);

    if (pthread_sigmask(SIG_UNBLOCK, &set, NULL)) {
        fprintf(stderr, "Não foi possível mascarar os sinais!");
        return 1;
    }

    return 0;
}

#define clp(argv, var) \
{ \
    char **endptr = NULL; \
    var = strtol (argv, endptr, 10); \
    if (endptr) { \
        fprintf (stderr, "Argumento inválido: %s\n", argv); \
    }

```

```

        return 1; \
    }\
}

int processarlc (int argc, char **argv) {
    if (argc > 1) {
        clp(argv[1], nlugares);
    } else {
        nlugares = 3;
    }

    if (argc > 2) {
        clp(argv[2], nbarbeiros);
    } else {
        nbarbeiros = 1;
    }

    if (argc > 3) {
        clp(argv[3], probabilidade);
    } else {
        probabilidade = 80;
    }

    return 0;
}

int main (int argc, char **argv) {
    int i;
    pthread_t *barbeiros;

    if (bloquearsinais() || processarlc(argc, argv)) {
        return 1;
    }

    if (sem_init (&slugares, 0, 0)) {
        fprintf (stderr, "Erro inicializando semáforo\n");
        return 1;
    }

    pthread_mutex_init (&mlugares, NULL);
    dlugares = 0;

    barbeiros = (pthread_t *)
        calloc (nbarbeiros, sizeof(pthread_t));

    if (!barbeiros) {

```

```

        fprintf (stderr, "Nenhum barbeiro virá trabalhar
                        hoje...\n");
        return 1;
    }

    for (i = 0 ; i < nbarbeiros ; i++) {
        fprintf (stdout, "O barbeiro %d chegou
                        para trabalhar.\n", i);

        pthread_create (&barbeiros[i], NULL, fbarbeiro, (void *)i);
    }

    liberarsinais();
    gerarclientes();

    return 0;
}

```

Esse programa recebe até três argumentos. O primeiro deles é a quantidade de clientes que poderão esperar, o número de barbeiros e a probabilidade do cliente entrar na loja. Portanto, para executar o programa com 3 lugares, 2 barbeiros e uma probabilidade de 90%, basta fazer:

```
$ ./barbeiro 3 2 90
```

Em nossa solução, a quantidade de clientes aguardando será representado pela variável *dlugares* (disponível lugares). Sei que ficou um pouco contra-intuitivo, pois seria mais adequado lugares ocupados, mas dá para sobreviver com isso. O acesso a essa variável é protegido pela *mutex mlugares*.

A geração de novos clientes e o trabalho do barbeiro foi simplificado ao máximo para que o exemplo fique bem didático. Mas para que tudo fique bem mais claro, vamos analisar a fundo as funções. A *fbarbeiro* faz o trabalho dos *N* barbeiros. A parte principal do código se encontra reproduzido abaixo:

```

if (sem_trywait (&slugares) == -1) {
    if (errno != EAGAIN){
        perror("Erro");
        break;
    }

    fprintf (stdout, "Barbeiro[%d]: Não tem nenhum
                    cliente...\n\tVou dormir...
                    zZzzZzZ...\n\n", bid);

    sem_wait (&slugares);
}

```

Na primeira linha, tentamos decrementar o semáforo *slugares*, o qual será usado para sinalizar a chegada de um novo cliente, com o uso da função *sem_trywait*. O motivo

para tal é que com ela podemos tentar decrementar o semáforo e realizar uma ação subsequente caso não seja possível fazê-lo no momento. Nossa ação, para nosso exemplo, é por o barbeiro para dormir, que não passa de imprimir uma mensagem na tela e pô-lo para esperar no semáforo com a função *sem_wait*. Então, enquanto não chegar um novo cliente, nossos barbeiros esperarão sentados e dormindo, pois ninguém é de ferro!

Assim que é acordado, o barbeiro procura fazer seu trabalho:

```
pthread_mutex_lock (&mlugares);  
q = --dlugares;  
pthread_mutex_unlock (&mlugares);
```

Simplesmente decrementamos a variável *dlugares* e guardamos seu valor atual na variável *q* para apresentar uma mensagem simples na tela. Lembrem-se, quando numa região crítica, devemos nos concentrar no trabalho a ser realizado e liberar a *mutex* tão cedo quanto possível! Outras *threads* estarão esperando para executar.

O trabalho realizado pela *thread* principal, gerar clientes, também é bem simples. Ela somente incrementa a variável *dlugares* a sinaliza no semáforo *slugares*. Mas para os clientes não ficarem chegando o tempo todo e não se possa observar a ação de dormir, é utilizado um valor randômico para determinar se um novo cliente entra na loja ou não.

Uma parte interessante do código diz respeito a recepção de sinais advindos do sistema operacional. As funções *bloquearsinais* e *liberarsinais* se encarregam da tarefa de evitar que alguns sinais cheguem as *threads* e liberar esses sinais para outras *threads*, respectivamente. O motivo para uso de tal abordagem é que o sistema operacional muitas vezes atrapalha o funcionamento dos semáforos, enviando sinais que as *threads* não estão preparadas para tratar, o que ocasiona alguns inconvenientes.

As outras funções são apenas aspectos não-funcionais, que não alteram em nada o propósito de passar uma noção mais prática dos mecanismos de sincronização.

4. Recomendações de Leituras

Apesar de não estarem explícitas no texto, essas referências foram muito importantes para a consolidação do mesmo:

1. **Mitchel, Oldham e Samuel.** *Advanced Linux Programming*. New Riders, 2001. www.advancedlinuxprogramming.com
2. **Marshall.** *Futher Threads Programming: Thread Attributes*. <http://www.cs.cf.ac.uk/Dave/C/node30.html>
3. **Maier.** *Threads Scheduling with pthreads under Linux and FreeBSD*. <http://www.net.t-labs.tu-berlin.de/gregor/tools/pthread-scheduling.html>
4. **Barney.** *POSIX Threads Programming*. <https://computing.lln.gov/tutorials/pthreads/>
5. **Lampkim.** *Pthreads: semi-FAQ Revision 5.2*. <http://www.cognitus.net/html/howto/pthreadSemiFAQ.html>
6. **Wikipedia.** *Dining Philosopher Problem*. http://en.wikipedia.org/wiki/Dining_philosophers_problem
7. **Wikipedia.** *Sleeping Barber Problem*. http://en.wikipedia.org/wiki/Sleeping_barber

Recomendo a leitura de cada uma delas. A referência 1 é a que mais recomendo. Ela possui versão impressa. Quem fizer pleno uso dos conhecimentos presentes aqui e no ALP, comprem o livro! É muito bom e vale cada centavo!

As próximas referências não tratam diretamente sobre processamento multithread ou paralelo, porém vale a referência pois apresentam conceitos importantes:

1. **Tanenbaum.** *Organização Estruturada de Computadores*, 5 ed. Prentice Hall, 2007. Nota: capítulo 8, Arquitetura de Computadores Paralelos.
2. **Tanenbaum.** *Sistemas Operacionais Modernos*, 2 ed. Prentice Hall, 2003.